Spark RDD



Spark RDD

❑ 复习

➢ Spark DataFrame

代码:

```
# 创建 DataFrame
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('Basics').getOrCreate()

df = spark.read.json('file:///home/… …/people.json')
df.show()


 # schema结构
 Schema = StructType([
                      StructField('age',IntegerType(),True),
                      StructField('name',StringType(),True),
 ])
```

```
# 可以自己设置schema
schema=
```

Spark RDD

❑ 复习

➢ Spark DataFrame

代码:

```
# 创建 DataFrame
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('Basics').getOrCreate()

df = spark.read.csv('appl_stock.csv',inferSchema=True,header=True)
df.show()
```

读取csv文件

Spark RDD

❑ 复习

➢ Spark SQL

代码:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("basic example").getOrCreate()

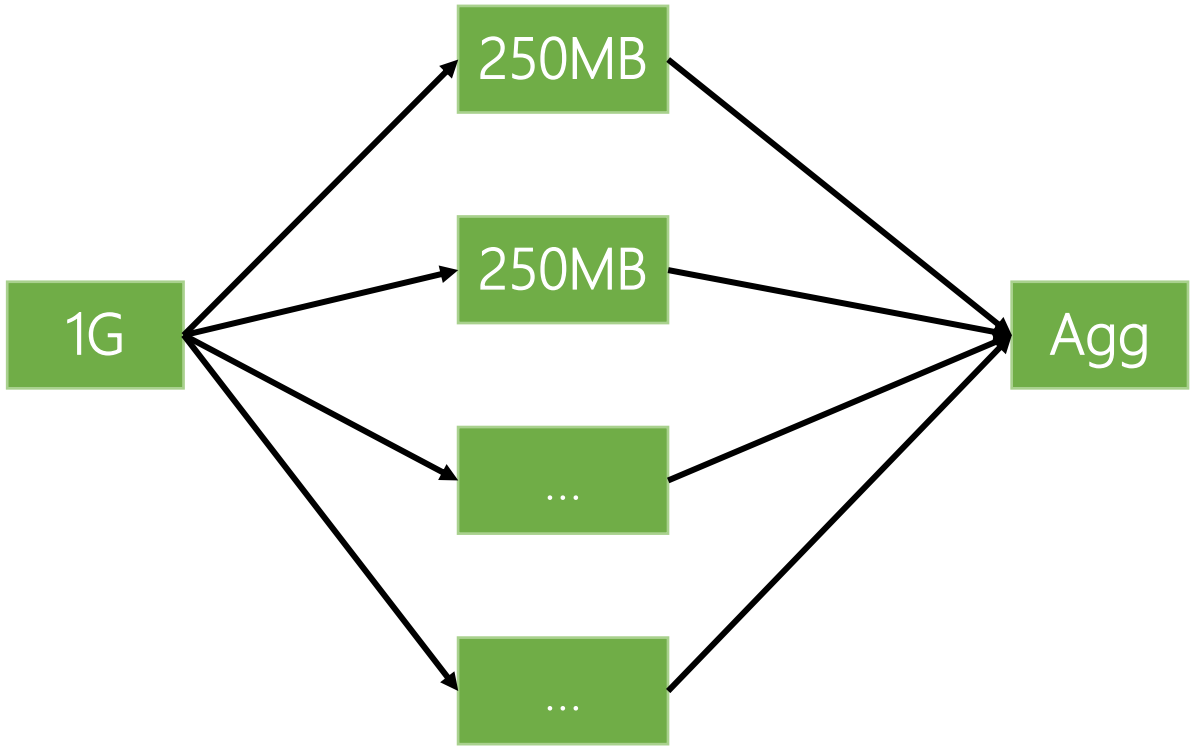
# spark is an existing SparkSession
df = spark.read.json("/people.json")

# Register the DataFrame as a SQL temporary view
df.createOrReplaceTempView("people")
sqlDF = spark.sql("SELECT * FROM people")
sqlDF.show()
```

```
+----+-------+
| age|   name|
+----+-------+
|null|Michael|
|  30|   Andy|
|  19| Justin|
+----+-------+
```

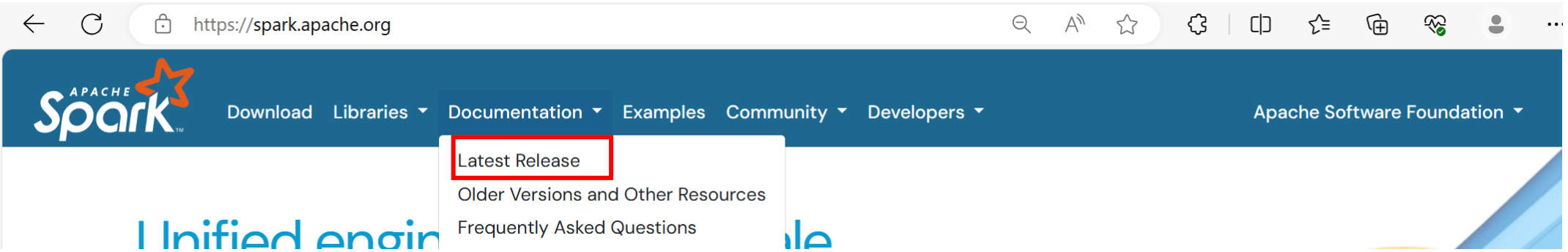
Spark RDD

❑ 并行运算



假设我们要对一个1GB的excel文件，求某一个特定列的平均值，如何进行?

Spark RDD



Python	SQL	Scala	Java	R

Launching on a Cluster

The Spark cluster mode overview explains the key concepts in running on a cluster. Spark can run both by itself, or over several existing cluster managers. It currently provides several options for deployment:

- Standalone Deploy Mode: simplest way to deploy Spark on a private cluster
- Apache Mesos (deprecated)
- Hadoop YARN
- Kubernetes

Spark RDD

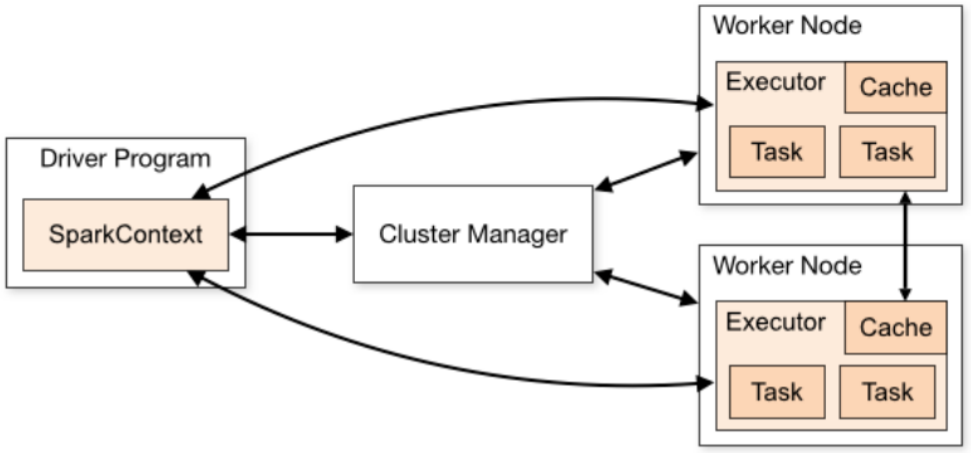
Cluster Mode Overview

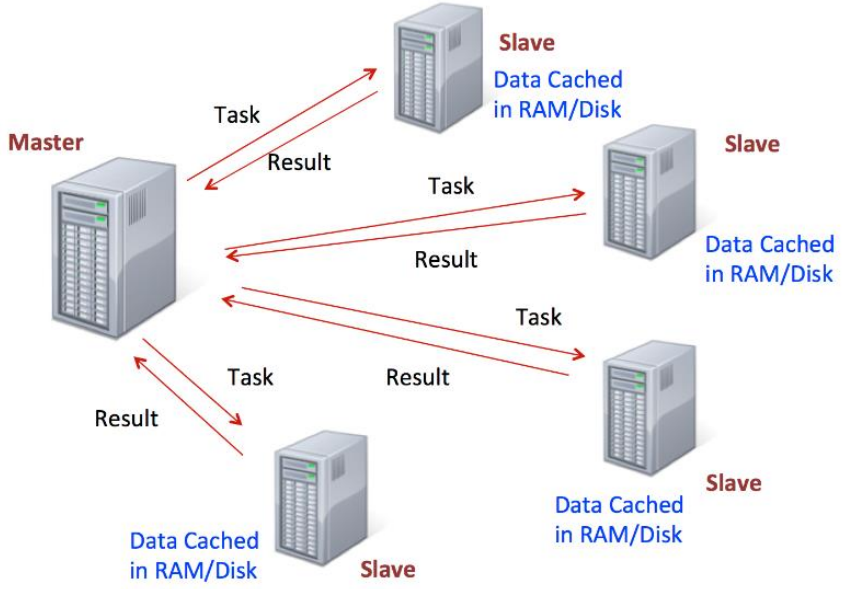
This document gives a short overview of how Spark runs on clusters, to make it easier to understand the components involved. Read through the application submission guide to learn about launching applications on a cluster.

Components

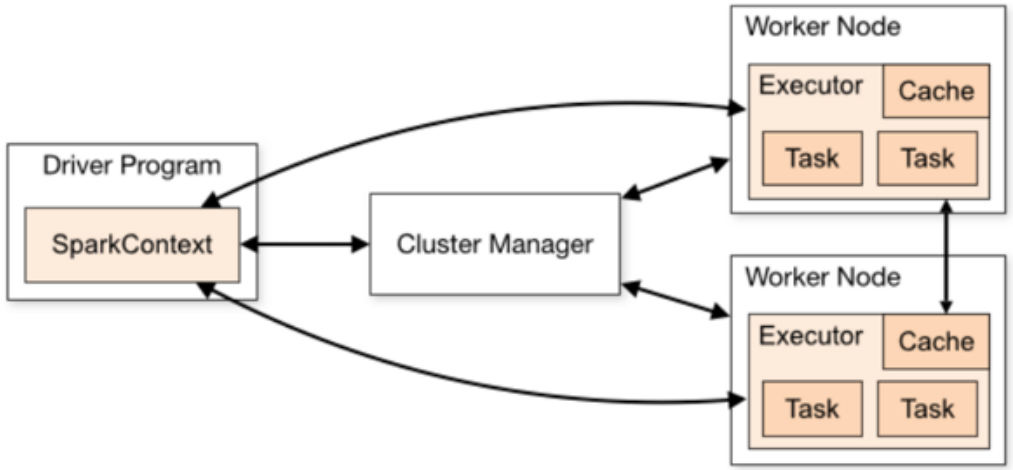
Spark applications run as independent sets of processes on a cluster, coordinated by the `SparkContext` object in your main program (called the *driver program*).

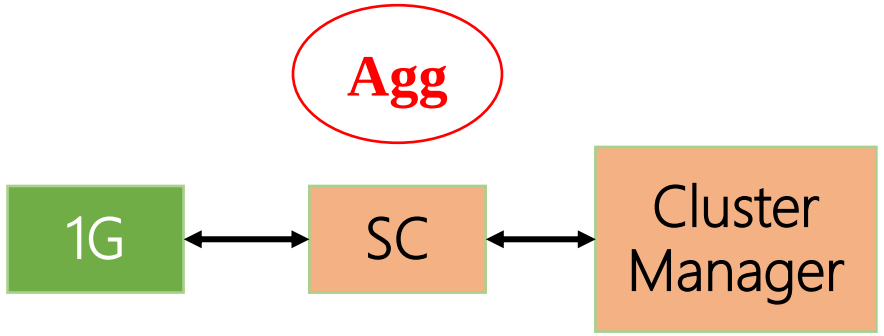
Specifically, to run on a cluster, the SparkContext can connect to several types of *cluster managers* (either Spark's own standalone cluster manager, Mesos, YARN or Kubernetes), which allocate resources across applications. Once connected, Spark acquires *executors* on nodes in the cluster, which are processes that run computations and store data for your application. Next, it sends your application code (defined by JAR or Python files passed to SparkContext) to the executors. Finally, SparkContext sends *tasks* to the executors to run.

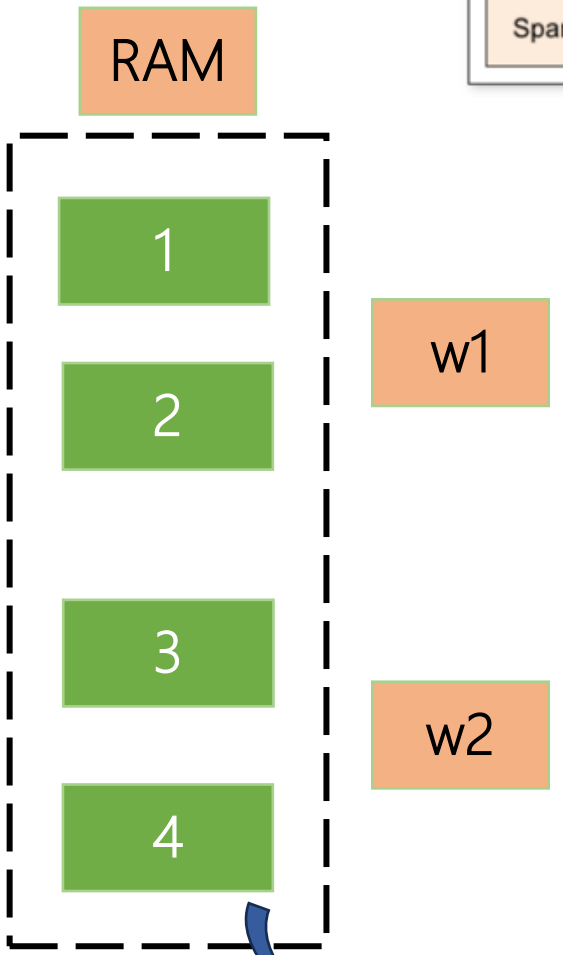


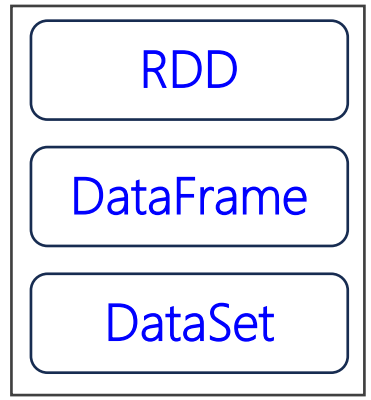


Spark RDD


Worker Node
Executor
Cache
Task
Task
Driver Program
SparkContext
Cluster Manager
Worker Node
Executor
Cache
Task
Task


Agg
1G
SC
Cluster Manager


RAM
1
2
3
4
w1
w2


RDD
DataFrame
DataSet


Agg

Spark RDD

❑ 基本概念

➢ RDD (Resilient Distributed Dataset)

RDD分布式数据集是Spark用来表达资料单元的一种概念，其资料结构具有**并行性、不可变性，容错性和惰性运算**的特色。

- 并行性：由于RDD是将数据切分为多个单位，因此能分布在多个运算资源上，所以达到并行运算的特性。
- 不可更动性 （Immutable）：正是因为将数据切分为多个单位，RDD 不允许我们对其数据本身做窜改，如果想要对数据做调整，只能透过操作产生新的 RDD。
- 容错性：RDD的容错机制主要依赖于它的不可变性和分布式性。由于RDD是不可变的，它的数据只能通过转换操作生成新的RDD，而不能直接修改原有的RDD。这种不可变性可以使得RDD在节点失败时可以快速地重新计算出丢失的数据，并生成新的RDD。此外，由于RDD是分布式的，它的数据可以被复制到多个节点上，从而在节点失败时可以从备份数据中重新生成丢失的数据。
- 惰性运算：在操作 RDD 的时候，通常会先有一系列的数据转换 （我们称为 **Transformation**），然后藉由某个特定运算生成结果（我们称为 **Action**），所谓惰性运算的概念是，无论资料如何进行转换，甚至转换了数次，只有在发生 Action 的当下，RDD 才会真的按照转换的需求开始动作然后生成结果。

Spark RDD

❑ 基本概念

➢ RDD (Resilient Distributed Dataset)

- ◆ There are two types of Spark operations:
  - •Transformations
  - •Actions
- ◆ Transformations are basically a recipe to follow.
- ◆ Actions actually perform what the recipe says to do and returns something back.

Spark RDD

❑ RDD编码

RDD Programming Guide

- Overview
- Linking with Spark
- Initializing Spark
  - Using the Shell
- Resilient Distributed Datasets (RDDs)
  - Parallelized Collections
  - External Datasets
  - RDD Operations
    - Basics
    - Passing Functions to Spark
    - Understanding closures
      - Example
      - Local vs. cluster modes
      - Printing elements of an RDD
    - Working with Key-Value Pairs
    - Transformations
    - Actions
    - Shuffle operations
      - Background
      - Performance Impact
  - RDD Persistence
    - Which Storage Level to Choose?
    - Removing Data
- Shared Variables
  - Broadcast Variables
  - Accumulators
- Deploying to a Cluster
- Launching Spark jobs from Java / Scala

◆ Transformations

- map
- flatmap
- filter
- distinct
- sample
- union, intersection

```
rdd = sc.parallelize([1,2,3,4])
rdd.map(lambda x: x*x)
```

```
def squar(x):
return x*x
```

◆ Actions

- collect
- count
- countByValue
- take
- top
- reduce
- … …

Spark RDD

❑ RDD – 代码部分

➢ RDD的创建和一些基本运算

```
# Link with spark
from pyspark import SparkContext, SparkConf

# Initializing Spark

conf = SparkConf().setAppName("rdd1").setMaster("local[*]")
sc = SparkContext(conf=conf)
print(sc)


# 目前使用的核心数cores
sc.defaultParallelism
```

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

在创建rdd之前，先创建一组随机数

```
# Crate random data
import random
randomlist = random.sample(range(0,40),10)
print(randomlist)
```

```
[34, 4, 14, 16, 17, 19, 39, 35, 13, 21]
```

```
# Create RDD:
rdd1 = sc.parallelize(randomlist,4) # sc is driver # 分区设置为4

rdd1.collect() #collect:返回所有数据集元素 collection is action , 如果数据很大，不使用collect
```

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

```
# Data distribution in partitions: (分区中的数据分布)

print(rdd1.getNumPartitions()) # 分区数
print(rdd1.glom().collect()) #  glom():得到分区格式的数据
print("The first two partitions:",rdd1.glom().take(2)) # 打印前两个分区数据
```

```
4
[[34, 4], [14, 16], [17, 19], [39, 35, 13, 21]]
The first two partitions: [[34, 4], [14, 16]]
```

Spark RDD

❑ RDD – 代码部分

➢ RDD的创建和一些基本运算

```
# 取前两个分区 take(2)
rdd1.glom().take(2)
```

```
[[34, 4], [14, 16]]
```

```
# 最后一个分区
rdd1.glom().collect()[-1]
```

```
[39, 35, 13, 21]
```

```
# count / first/ top 操作
# 注: top按顺序排列之后，取前（）个
# top(2):这里只有10个值，按顺序排列取出前2个;

print("Count:",rdd1.count())
print('First element:',rdd1.first())
print("Top two elements:",rdd1.top(2))
```

```
Count: 10
First element: 34
Top two elements: [39, 35]
```

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

这里需要说明：在driver (sc)上的运行的操作需要小心，因为它不进行并行运算。
对于计算，我们使用transformations; 对于信息打印，我们使用actions

```
# distinct 去重 (transformation)
rdd1.distinct()
```

```
PythonRDD[14] at RDD at PythonRDD.scala:53
```

```
rdd1.distinct().collect()
```

```
[4, 16, 17, 13, 21, 34, 14, 19, 39, 35]
```

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

```
# map

def myfunc(item):
    return (item+1) * 3

rdd_map1 = rdd1.map(myfunc)

rdd_map2 = rdd1.map(lambda x:(x+1)*3)

print(rdd_map1.collect())
print(rdd_map2.collect())
```

```
[105, 15, 45, 51, 54, 60, 120, 108, 42, 66]
[105, 15, 45, 51, 54, 60, 120, 108, 42, 66]
```

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

```
# filter
rdd_filter = rdd1.filter(lambda x: x%3 ==0)
print("filter之后的元素: ")
print(rdd_filter.glom().collect())
```

```
filter之后的元素：
[[], [], [], [39, 21]]
```

```
# 改变分区数repartition

rdd2 = rdd1.repartition(3)
rdd2.getNumPartitions()
```

```
3
```

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

```
# flatMap 一对多
rdd_flatmap = rdd1.flatMap(lambda x:[x+1,x+2,x+3])
rdd_flatmap.glom().collect()
```

```
[[35, 36, 37, 5, 6, 7],
 [15, 16, 17, 17, 18, 19],
 [18, 19, 20, 20, 21, 22],
 [40, 41, 42, 36, 37, 38, 14, 15, 16, 22, 23, 24]]
```

```
# reduce
rdd_flatmap.reduce(lambda x,y:x+y)
```

```
696
```

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

```
# 一些统计值：最大、最小，平均
print(rdd1.max(),rdd1.min(),rdd1.mean(),round(rdd1.stdev(),2),rdd1.sum())
```

```
39 4 21.2 10.66 212
```

```
# 对每个分区进行求和 - mapPartitions
def myfunc(partition):
    sum = 0
    for item in partition:
        sum = sum + item
    yield sum

rdd_mappartition = rdd1.mapPartitions(myfunc)
print("对每个分区求和的结果是:\n",rdd_mappartition.glom().collect())
```

```
对每个分区求和的结果是:
 [[38], [30], [36], [108]]
```

Spark RDD

❑ RDD – 代码部分

➢ RDD的创建和一些基本运算

在 PySpark 中，map() 和 mapPartitions() 是两个转换操作，用于以分布式方式将函数应用于 RDD 的每个元素。 但是，也存在以下一些差异：

- map()是一个转换操作，它将指定的函数应用于RDD的每个元素并返回一个新的RDD。 传递给map()的函数**单独应用于RDD的每个元素**。 它一次只对一个元素进行操作，当函数开销较高或需要外部资源时，速度可能会变慢。
- mapPartitions()是一个转换操作，它将指定的函数应用于RDD的每个分区并返回一个新的RDD。 传递给 mapPartitions() 的函数**作为一个整体应用于每个分区，而不是单个元素**。 当函数具有较高开销或需要外部资源时，它可以更有效，因为它通过一次处理多个元素来减少函数调用的开销。

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

```
# 并集 union
rdd2 = sc.parallelize([1,14,20,39,28,11,13,3],3)
rdd_union = rdd1.union(rdd2)
print("rdd1和rdd2的并集:\n",rdd_union.glom().collect())
print("rdd1和rdd2的合并后的分区数:",rdd_union.getNumPartitions())
```

```
rdd1和rdd2的并集:
 [[34, 4], [14, 16], [17, 19], [39, 35, 13, 21], [1, 14], [20, 39], [28, 11, 13, 3]]
rdd1和rdd2的合并后的分区数: 7
```

```
# 交集 Intersection
rdd_intersection = rdd1.intersection(rdd2)
print("rdd1和rdd2的交集: \n",rdd_intersection.glom().collect())
```

```
rdd1和rdd2的交集：
 [[14], [], [], [], [39], [], [13]]
```

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

```
# 交集为空的分区数
counter = 0
for item in rdd_intersection.glom().collect():
    if len(item) == 0:
        counter += 1
print("交集为空的分区数:",counter)
```

```
交集为空的分区数: 4
```

```
# 使用coalecse减少分区数量
rdd_intersection_reduce = rdd_intersection.coalesce(1)
print("将分区数减少到1个分区后，rdd1和rdd2的交集:
",rdd_intersection_reduce.glom().collect())
```

```
将分区数减少到1个分区后，rdd1和rdd2的交集： [[14, 39, 13]]
```

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

通常，我们对大数据，先进行：map，filter... 进行一系列的transformation（转换）操作，到大数据-->变得很小如mb大小），然后我们再进行action操作，如takeSample(),takeOrdered(),collect()等

```
# 取样 takeSample
print("取样操作:",rdd1.takeSample(False,5)) # False: with replacement is : false
```

```
取样操作: [35, 14, 34, 4, 21]
```

```
# takeOrdered
rdd1.takeOrdered(5) # 默认升序
```

```
[4, 13, 14, 16, 17]
```

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

```
#  降序
rdd1.takeOrdered(5,key=lambda x:-x)
```

```
[39, 35, 34, 21, 19]
```

```
# reduce
rdd1.reduce(lambda x,y:x-y)
```

```
64
```

reduce是一种聚合运算具体实施：
rdd1: [[34,4],[14,16],[17,19],[39,35,13,21]]
Reduce的运算是将表达式一个个运算到每个分区上，再做整体运算，这里的ruduce运算是：x-y：
1>
34-4 = 30
14-16 = -2
17-19 = -2
[(39-35) -13]-21 = -30
2>
[(30-（-2）-（-2）]-（-30） = 64

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

```
# reduceByKey() 和 sortByKey()
rdd3 =
sc.parallelize([(1,4),(7,10),(5,7),(1,12),(7,12),(7,1),(9,1),(7,4)],2)
print("先定义一组数据（有2个分区）:",rdd3.glom().collect())
```

```
先定义一组数据（有2个分区）: [[(1, 4), (7, 10), (5, 7), (1, 12)], [(7, 12), (7, 1), (9, 1), (7, 4)]]
```

```
# key- value (visualization)
import pandas as pd
Counter = pd.DataFrame({"key":rdd3.keys().collect(),
                        "values":rdd3.values().collect()
                       })
print(Counter)
```

	key	values
0	1	4
1	7	10
2	5	7
3	1	12
4	7	12
5	7	1
6	9	1
7	7	4

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

```
# reduceByKey
rdd_rbk = rdd3.reduceByKey(lambda x,y:x+y)
print("使用聚合函数reduceByKey()后:",rdd_rbk.collect())
```

```
使用聚合函数reduceByKey()后: [(1, 16), (7, 27), (5, 7), (9, 1)]
```

```
# sortByKey()
rdd_rbk.sortByKey().collect()
```

```
[(1, 16), (5, 7), (7, 27), (9, 1)]
```

```
rdd_rbk.sortByKey(True).collect()
```

```
[(1, 16), (5, 7), (7, 27), (9, 1)]
```

```
rdd_rbk.sortByKey(False).collect()
```

```
[(9, 1), (7, 27), (5, 7), (1, 16)]
```

Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

```
# countByKey()
print("rdd3中每个key出现的次数:{dict的形式}",rdd3.countByKey())
```

```
rdd3中每个key出现的次数:{dict的形式} defaultdict(<class 'int'>, {1: 2, 7: 4, 5: 1, 9: 1})
```

```
# countByKey()
print("rdd3中每个key出现的次数:(元组的形式)",rdd3.countByKey().items())
```

```
rdd3中每个key出现的次数:(元组的形式) dict_items([(1, 2), (7, 4), (5, 1), (9, 1)])
```

Spark RDD

❑ RDD – 代码部分

➢ RDD的创建和一些基本运算

```
# groupByKey()
rdd_group = rdd3.groupByKey()
rdd_group.collect()  # -- how to print
```

```
[(1, <pyspark.resultiterable.ResultIterable at 0x7fa8f81ae730>),
 (7, <pyspark.resultiterable.ResultIterable at 0x7fa8d7ebd1f0>),
 (5, <pyspark.resultiterable.ResultIterable at 0x7fa8d7ebd250>),
 (9, <pyspark.resultiterable.ResultIterable at 0x7fa8d7ebd2e0>)]
```

```
for item in rdd_group.collect():
    print(item[0],[values for values in item[1]])
```

```
1 [4, 12]
7 [10, 12, 1, 4]
5 [7]
9 [1]
```

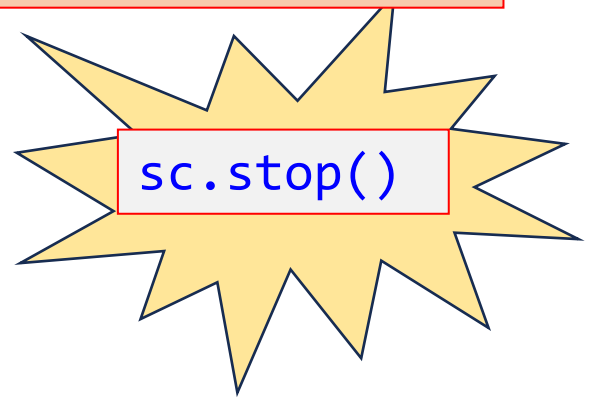
Spark RDD

- RDD – 代码部分
  - RDD的创建和一些基本运算

```
# lookup()
rdd3.lookup(7) # key=7的values
```

```
[10, 12, 1, 4]
```

在运算完成之后将spark关闭

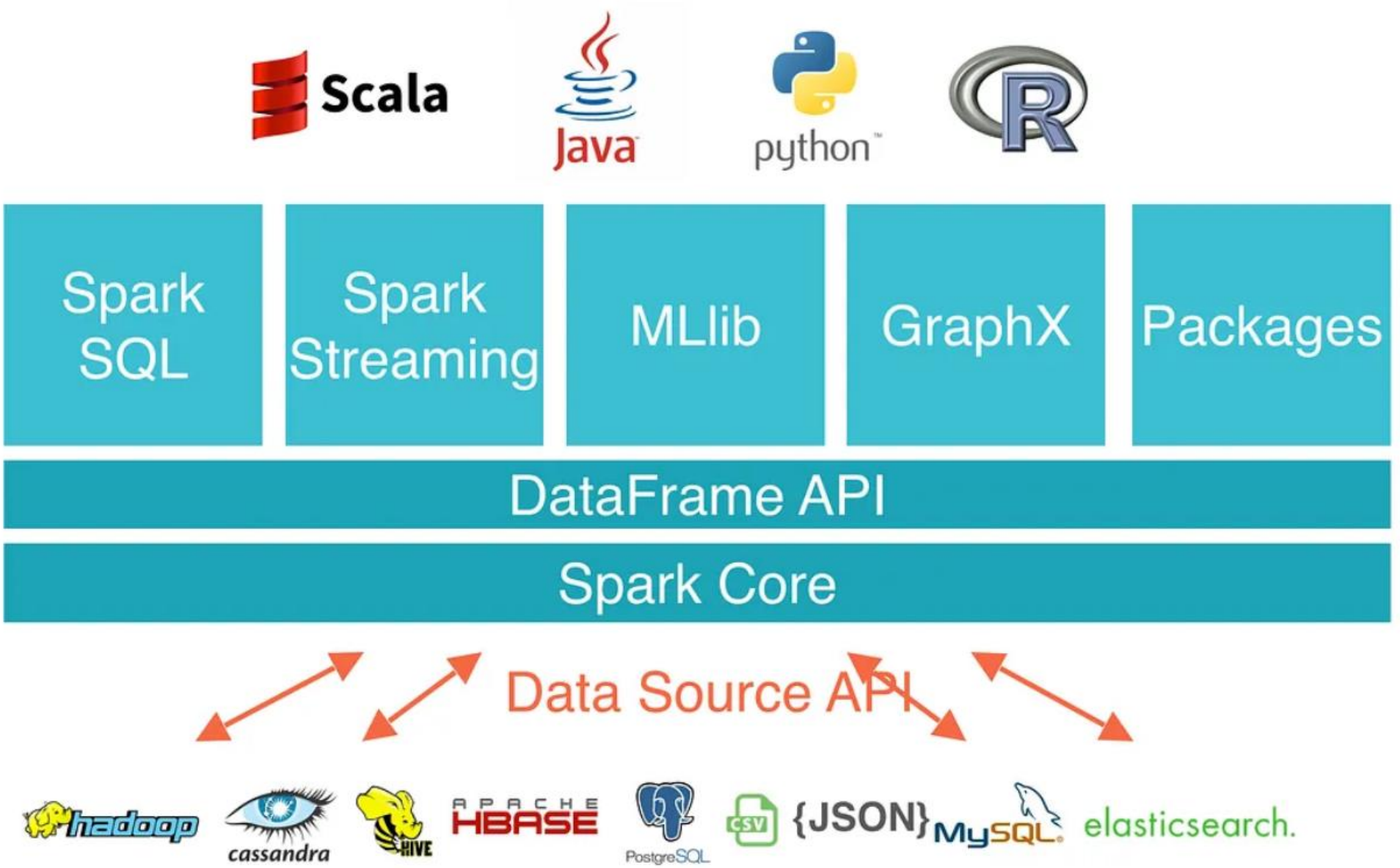


缓存:默认情况下，每次对每个转换后的 RDD 运行操作时，可能会重新计算它。但是，您也可以使用 persist（或缓存）方法将 RDD 保留在内存中，在这种情况下，Spark 会将元素保留在集群上，以便下次查询时更快地访问

```
rdd3.persist() # rdd3.cache()
from pyspark import StorageLevel
rdd3.persist(StorageLevel.MEMORY_AND_DISK) # 设置此 RDD 的存储级别,默认为 MEMORY_ONLY
```

https://spark.apache.org/docs/latest/rdd-programming-guide.html#rdd-persistence

Spark RDD


Scala
Java
python
R
Spark SQL
Spark Streaming
MLlib
GraphX
Packages
DataFrame API
Spark Core
Data Source API
hadoop
cassandra
HIVE
APACHE HBASE
PostgreSQL
CSV
{JSON}
MySQL
elasticsearch.